
Linear Regression

Contact Session 3 — Module 3

*Predicting a number by drawing the best line. We build the whole idea slowly, in plain words,
and only meet each equation after you already understand it.*

- | | |
|------------------|------------------|
| ■ Intuition | ■ Analogy |
| ■ Worked example | ■ Solved problem |
| ■ Common pitfall | ■ Key takeaways |

Contents

1	Start here: what are we trying to do?	3
2	The simplest idea in the world: draw a line	4
3	What the line's two numbers mean	5
4	What if we know more than one thing about the house?	6
5	How good is a line? Meet the “miss”	7
6	Turning many misses into one score	8
7	“Learning” is just: find the line with the smallest score	9
8	Way 1: a formula that jumps straight to the bottom	10
9	Way 2: roll downhill, one small step at a time	11
10	Watch one step happen (a fully worked example)	13
11	One-shot formula or downhill walk — which should I use?	15
12	How much data per step? (three flavours)	16
13	What if the dots don't follow a straight line?	17
14	The Goldilocks problem: too simple, too wiggly, just right	18
15	Keeping the line sensible: regularisation	19
16	Did it actually work? A simple report card	20
17	The whole story on one page	21

1 Start here: what are we trying to do?

Imagine you want to guess the price of a house. You know one thing about it: its size. And you have a notebook of past sales — for each old house, its size and the price it sold for. Can that notebook help you guess the price of a brand-new house you have never seen?

That is the entire problem. We have an **input** (size) and we want to guess an **output** (price). When the output is a *number* — a price, a temperature, a marks total — we call this **regression**. (If the output were a yes/no label instead, we'd call it classification, which is the next session.)

★ The one-line summary

Regression means: look at past examples, spot the pattern, and use it to put a number on something new. Everything below is just *how* we do that carefully.

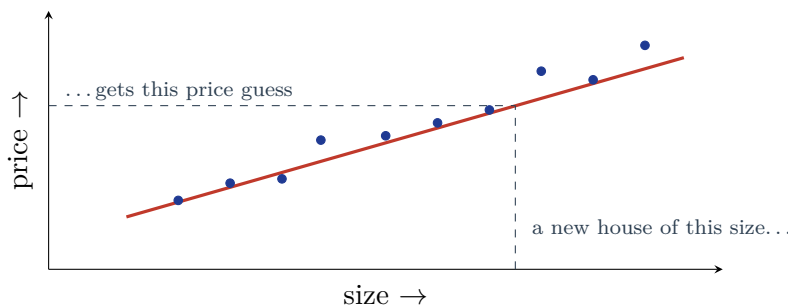
2 The simplest idea in the world: draw a line

Let us put the past sales on a graph. Size runs along the bottom, price runs up the side. Each old house becomes one dot.



Look at the dots. As size goes up, price goes up — the dots roughly climb from bottom-left to top-right. So here is the simplest possible idea: **draw one straight line through the middle of the dots**. Then, to guess the price of a new house, find its size along the bottom, go straight up to the line, and read the price off the side.

That's it. The line *is* our predictor.



~> A line is just a rule of thumb

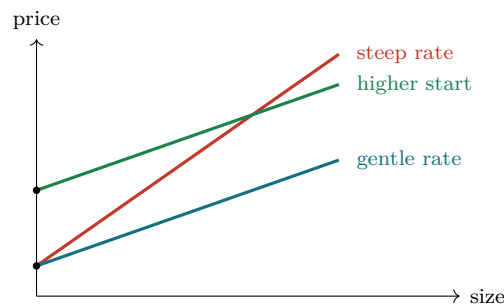
“Start at 50 lakh, and add about 5,000 for every extra square foot.” That sentence *is* a straight line. A line is nothing fancier than a steady rule: start somewhere, then go up (or down) by the same amount for every step you take to the right.

3 What the line's two numbers mean

Every straight line is pinned down by just **two numbers**:

- where the line **starts** — the price it gives when size is zero. Call it the *starting value*.
- how **steep** it is — how much the price climbs for each extra unit of size. Call it the *rate*.

Change those two numbers and you get a different line. A bigger starting value lifts the whole line up. A bigger rate tilts it steeper.



Now, mathematicians like to write this line with symbols. Here it is — but please don't let the symbols scare you, because you already understand what they say:

$$\hat{y} = \theta_0 + \theta_1 x.$$

Read it slowly, in plain words:

- $\hat{y}$ (say “y-hat”) is our **guess** for the price.
- x is the **size** we feed in.
- θ_0 (“theta-zero”) is the **starting value**.
- θ_1 (“theta-one”) is the **rate**.

So the scary-looking equation is really just:

$$\text{guess} = \text{starting value} + \text{rate} \times \text{size}.$$

You understood it before you ever saw a Greek letter.

★ The only things we get to choose

The size x is given to us by the house. Our *guess* comes out automatically. The only things *we* pick are the two knobs θ_0 and θ_1 . So “learning” will turn out to mean one simple thing: **find good settings for the knobs.**

4 What if we know more than one thing about the house?

Real houses have a size, an age, a number of bedrooms, a location. Easy — we just give *each* fact its own rate knob and add everything up:

$$\text{guess} = \text{start} + (\text{rate}_1 \times \text{size}) + (\text{rate}_2 \times \text{age}) + (\text{rate}_3 \times \text{bedrooms}) + \dots$$

Each feature gets a knob that says how strongly that fact pushes the price. A *positive* rate pushes the price up (bigger size $\rightarrow$ higher price). A *negative* rate pulls it down (older house $\rightarrow$ lower price, so age gets a negative rate). In symbols this is just

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d,$$

where $x_1, x_2, \dots$ are the different facts and $\theta_1, \theta_2, \dots$ are their rates. Same idea, more knobs.

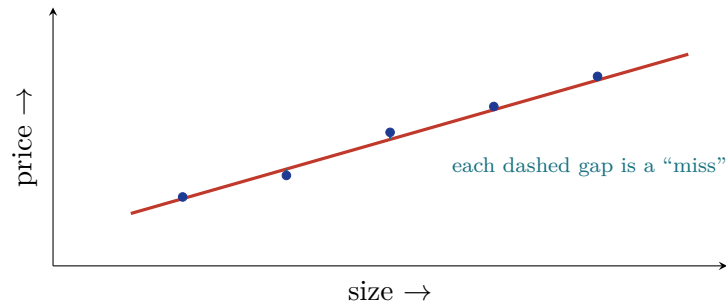
★ “Linear” demystified

The word **linear** sounds technical, but it only means one thing here: we *add things up*. Each fact is multiplied by its own rate, and the results are added together. Nothing is squared, nothing is multiplied with another fact, nothing is twisted. Multiply-and-add. That is all “linear” means.

5 How good is a line? Meet the “miss”

We could draw a thousand different lines through those dots. Which one is best? To even ask that, we first need a way to measure how *wrong* a line is.

Take one house. Its real price was 60. Our line guessed 55. We were off by 5. That gap — *real price minus our guess* — is the line’s **miss** on that house. (Textbooks call it the *residual*, but “the miss” or “the gap” says it perfectly.)



A *good* line has small gaps on most houses. A *bad* line has big gaps. So judging a line really comes down to one question: **how big are all the gaps, taken together?**

6 Turning many misses into one score

We have one gap per house — maybe hundreds of them. We want a *single* number that says “this line is this bad.” How do we squash hundreds of gaps into one score?

First try: just add the gaps. Trouble: some gaps are above the line (we guessed too low, a $+$) and some below (we guessed too high, a $-$). A $+5$ and a -5 cancel to 0, making a terrible line look perfect. We cannot let plus and minus cancel.

The fix: square each gap before adding. Squaring a number just means multiplying it by itself. A -5 becomes 25; a $+5$ also becomes 25. Now everything is positive and nothing cancels. Squaring has a nice bonus too: a big miss of 10 becomes 100, which hurts far more than two small misses of 5 (that’s only $25 + 25 = 50$). So squaring quietly says “*I really hate big mistakes*” — usually exactly what we want.

Then average (add them up, divide by the number of houses) so the score doesn’t balloon just because we happen to have more houses.

That recipe has a name: the **Mean Squared Error**, or **MSE**. Read the name backwards: *Error* = the gaps; *Squared* = each gap times itself; *Mean* = average. In symbols:

$$J = \frac{1}{n} \sum_{i=1}^n (\text{real}_i - \text{guess}_i)^2.$$

Translate it left to right: for every house i , take (real – guess), square it, add them all up — that curly $\sum$ symbol just means “add up everything” — and divide by the number of houses n to get the average. The letter J is simply a name for this *badness score*. **Smaller J means a better line.**

★ A tiny note on a $\frac{1}{2}$ you’ll see

Many books write $\frac{1}{2n}$ instead of $\frac{1}{n}$ out front. The extra $\frac{1}{2}$ is pure convenience — later, when we do a bit of calculus, a 2 pops out and cancels it neatly. It changes nothing about which line is best, so don’t give it a second thought.

↪ Springs pulling a rod

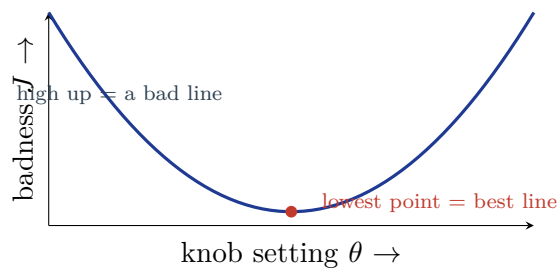
Picture each dot tied to the line by a little vertical *spring*. A stretched spring stores energy, and that energy grows with the *square* of the stretch. Adding up squared gaps is adding up spring energy. The line settles where the total pull is gentlest — the balance point. That balance point is the best line.

7 “Learning” is just: find the line with the smallest score

Here is the punchline of the whole chapter. Training a linear-regression model means exactly one thing:

Out of all possible lines, **find the one whose badness score J (the MSE) is smallest.** That line is our answer. Everything else in this chapter is just *how* to hunt for it.

To picture the hunt, imagine a new graph. Along the bottom we vary the knob settings (the θ 's). Going up, we plot the badness score J for each setting. The shape we get is a **bowl**.



Every knob setting is a spot on the bowl, and its height is how bad that line is. The best line sits at the very **bottom** of the bowl. So our whole job is: *get to the bottom*.

★ A lucky gift

For linear regression the bowl has only *one* bottom — no little side-dips to fall into and get stuck. So if we simply keep heading downhill, we are *guaranteed* to reach the single best line. Many fancier models are not this lucky; we should enjoy it while we can.

There are two ways to reach the bottom. One jumps there in a single leap; the other walks down step by step.

8 Way 1: a formula that jumps straight to the bottom

Because this bowl is so simple and smooth, mathematicians worked out a *formula* that lands exactly on the bottom in one shot — no searching at all. It is called the **normal equations**:

$$\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Don't worry about computing this by hand — you never will; the computer does the matrix arithmetic. What matters is the *idea*: there is a direct formula, you feed in your data ($\mathbf{X}$ holds all the house facts, $\mathbf{y}$ holds all the real prices), and out pops the best knob settings instantly.

★ When is the one-shot formula a good idea?

It is wonderful when you don't have too many facts (features) per house. The catch is hidden in that “ -1 ”: undoing the $\mathbf{X}^\top \mathbf{X}$ block gets slow and shaky when there are very many features. In that case we switch to Way 2.

9 Way 2: roll downhill, one small step at a time

When the one-shot formula is too expensive, we find the bottom the way a ball would: we let it *roll downhill*.

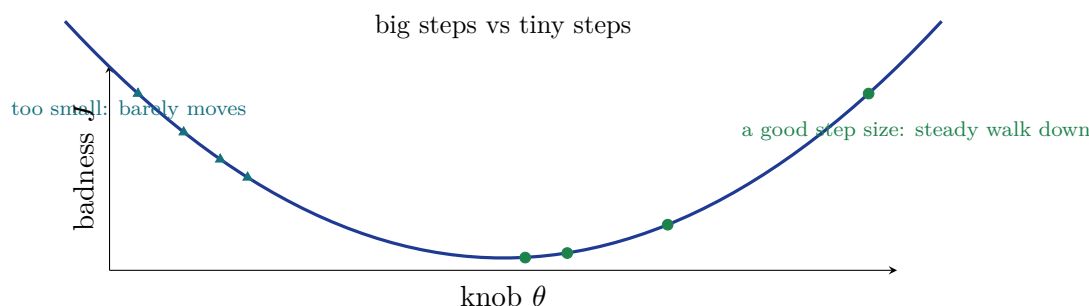
↪ Walking down a hill in thick fog

You're on a hillside and want the valley floor, but the fog is so thick you can't see it. All you can feel is the slope under your feet. So you take a step in the steepest downhill direction, feel again, step again, and repeat. Big steps get you down fast but risk striding right over the valley and up the other side. Tiny steps are safe but slow. This careful downhill walk is the whole method.

The method even has a name: **gradient descent** (a grand title for “keep stepping downhill”). The recipe:

1. Start with any line at all (any knob settings).
2. Figure out which way is downhill.
3. Nudge the knobs a small step that way.
4. Repeat until you stop going down (you've reached the bottom).

How big is each step? That's the **learning rate**, written α (“alpha”). It is simply your step size.



What does “nudge the knobs” actually look like? For each knob, the rule is:

$$\text{new knob} = \text{old knob} - (\text{step size}) \times (\text{how wrong we were, in that knob's direction}).$$

“How wrong we were” is just the *average gap*, paying attention to the feature that knob controls. In symbols it reads

$$\theta_j \leftarrow \theta_j - \alpha \cdot \frac{1}{n} \sum_{i=1}^n (\text{guess}_i - \text{real}_i) x_{ij},$$

which is exactly the sentence above, written tightly. We will now watch this happen with real numbers.

★ Why “gap $\times$ feature” is the downhill direction

You might wonder where that exact recipe comes from. “Downhill” means the slope of the badness score, and the badness was the *squared* gap. A handy fact about squaring: the slope of “something squared” is just “twice that something.” So the slope of each squared gap points along (guess – real) — the gap itself — and the x_{ij} rides along because that’s the feature this particular knob multiplies. (That stray “twice” is the very 2 the convenient $\frac{1}{2}$ in the cost quietly cancels.) You don’t need the calculus to use the rule, but now you can see it isn’t pulled from thin air: *the gap is the slope*.

! If the step size is wrong

Too *big* a step and the ball leaps past the bottom and bounces *higher* each time — the badness score goes up instead of down. So if you ever see J rising, your first move is to *shrink* the step size α . Too *small* and it works but crawls. People usually try a few values like 0.001, 0.01, 0.1 and watch which one brings J down smoothly.

10 Watch one step happen (a fully worked example)

Let's actually turn the handle once and watch the knobs move. We'll predict a heart-disease risk score from two facts about a patient: their BMI and their blood pressure.

Σ One step of downhill walking

Our line is $\text{guess} = w_0 + w_1 \cdot \text{BMI} + w_2 \cdot \text{Pressure}$. (We're using w for the knobs here; it's the same idea as θ .) We start the knobs at $w_0 = 5$, $w_1 = -0.03$, $w_2 = -0.03$, with step size $\alpha = 0.02$ and $n = 3$ patients.

Patient	BMI	Pressure	real risk
1	35	80	1.81
2	25	80	1.22
3	30	100	1.71

Step 1 — make a guess for each patient, and see how far off we are. Using $\text{guess} = 5 - 0.03 \text{ BMI} - 0.03 \text{ Pressure}$, and $\text{gap} = \text{guess} - \text{real}$:

$$\text{Patient 1: } 5 - 1.05 - 2.40 = 1.55, \quad \text{gap} = 1.55 - 1.81 = -0.26,$$

$$\text{Patient 2: } 5 - 0.75 - 2.40 = 1.85, \quad \text{gap} = 1.85 - 1.22 = +0.63,$$

$$\text{Patient 3: } 5 - 0.90 - 3.00 = 1.10, \quad \text{gap} = 1.10 - 1.71 = -0.61.$$

Step 2 — add up the gaps (three sums, one per knob). For the starting knob w_0 we just add the gaps. For each rate knob we add the gaps but weight each by the fact it controls:

$$S_0 = -0.26 + 0.63 - 0.61 = -0.24,$$

$$S_1 = (-0.26)(35) + (0.63)(25) + (-0.61)(30) = -11.65,$$

$$S_2 = (-0.26)(80) + (0.63)(80) + (-0.61)(100) = -31.40.$$

Step 3 — nudge each knob. The nudge is $\frac{\alpha}{n} = \frac{0.02}{3} = 0.006667$ times the sum:

$$w_0 = 5 - 0.006667(-0.24) = \mathbf{5.0016},$$

$$w_1 = -0.03 - 0.006667(-11.65) = \mathbf{0.0477},$$

$$w_2 = -0.03 - 0.006667(-31.40) = \mathbf{0.1793}.$$

New line: $\text{guess} = 5.0016 + 0.0477 \text{ BMI} + 0.1793 \text{ Pressure}$.

★ What just happened, in words

The starting knob barely moved — it was already about right. But both rate knobs *flipped* from -0.03 to a positive number. That makes sense: we started by saying “higher BMI lowers risk,” which is backwards, so a single downhill step pushed those knobs the correct

way. Do this hundreds of times and the line keeps getting better until it can't improve any more. That repetition *is* the learning.

11 One-shot formula or downhill walk — which should I use?

One-shot formula	Downhill walk (gradient descent)
no step size to pick, no repeating	you choose a step size and repeat
gives the exact best line at once	sneaks up on the best line
slows down with very many features	happily handles huge data
best when you have few features	best when data is large

★ One more reason the downhill walk wins in practice

If your features have wildly different sizes (BMI around 30, but income in the lakhs), the bowl turns into a long, narrow ravine and the ball zig-zags slowly to the bottom. **Scaling** the features to similar ranges (Module 2) makes the bowl nicely round again, so the ball rolls straight in. This is the practical reason we bothered with feature scaling earlier.

12 How much data per step? (three flavours)

The downhill walk needs to know the slope, which it gets by looking at the data. How *much* data it looks at before each step gives three flavours:

- **All of it each step** (“batch”) — very steady steps, but slow when data is huge.
- **One example each step** (“stochastic”) — fast and jumpy; it never fully settles, just hovers near the bottom.
- **A small handful each step** (“mini-batch”) — the happy middle, and what almost everyone actually uses.

13 What if the dots don't follow a straight line?

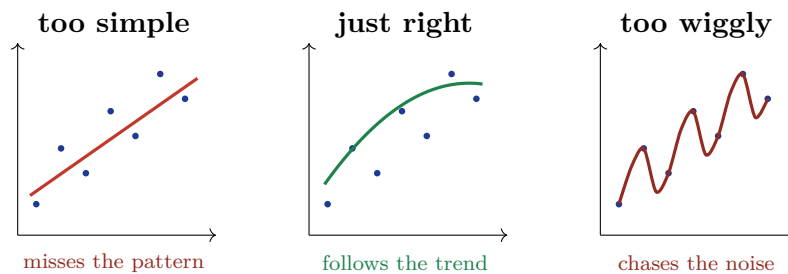
Sometimes the dots clearly *bend* — a curve fits them better than any straight line. Are we stuck? Not at all. Here's a neat trick: feed the model a new “fact” that is just the size *multiplied by itself* (size-squared), and maybe size-cubed too. The model treats these as ordinary extra facts, gives each its own rate knob, and adds them up as before. The result can bend into a curve.

★ Still “adding up”, still easy

The lovely part: even though the *picture* is now a curve, the *method* hasn't changed one bit. We still just multiply each fact by a knob and add them up; we still slide to the bottom of a bowl. We only changed *what we feed in*. So curves cost us no new machinery — only a richer set of ingredients.

14 The Goldilocks problem: too simple, too wiggly, just right

If we throw in too few ingredients, the model is too stiff to follow the real pattern. If we throw in too many, it starts chasing every tiny wobble — including random noise that means nothing. We want the line that’s *just right*.



The two ways to be wrong even have names. **Too simple** is called high *bias* (the model is stubborn and won’t bend to the data). **Too wiggly** is called high *variance* (the model is so jumpy that it changes wildly if you give it slightly different data).

↪ The student who memorised vs. the student who understood

A student who *memorises* last year’s exam (too wiggly) aces a re-run but flunks a fresh paper. A student who only learned “study hard” (too simple) is too vague to answer anything specific. The student who learned the *ideas* lands in the middle — general enough to handle new questions, sharp enough to be useful. Our model is chasing that same middle ground.

How do we land in the middle? A few simple levers: get *more data* (it pins the wiggles down), use *fewer ingredients*, *stop training early* before the model starts memorising, or use the next idea — regularisation.

15 Keeping the line sensible: regularisation

When a model gets too wiggly, it's usually because some knobs have grown into huge numbers, swinging the curve violently. **Regularisation** is a gentle fix: we add a small penalty for big knobs to the badness score. Now the model has to balance two wishes — “fit the dots” *and* “keep the knobs small” — so it stops over-reacting.

There are two popular penalties:

- **Ridge** (the gentle one): pushes every knob *toward* zero, making them all small but keeping them. Smooth and well-behaved.
- **Lasso** (the ruthless one): can push a knob *all the way to* zero, switching that fact off entirely. Handy when you suspect some facts are useless — it does feature-cleanup for you.

(There's also **Elastic Net**, which simply mixes the two.) A dial called λ sets how hard we push: $\lambda = 0$ means “no penalty” (back to plain regression), a big λ means “keep knobs tiny” (very smooth, maybe too smooth). We pick a sensible middle value by testing on held-back data.

★ Two styles of penalty, in one breath

Ridge tells every knob, “please speak softly.” Lasso tells the useless knobs, “be quiet completely.” That's why Lasso doubles as a feature cleaner — it can silence facts that don't help.

16 Did it actually work? A simple report card

Once we have a line, we test it on houses it has *never seen* (a held-back “test set”). Doing well on the training dots isn’t enough — a memoriser does that too. The usual report-card number is called **R-squared** (R^2):

- $R^2 = 1$ means a perfect fit — the line explains everything.
- $R^2 = 0$ means the line is no better than just guessing the average price every time.
- Negative means it’s actually *worse* than guessing the average.

In words, R^2 answers: “of all the ups and downs in price, what fraction did my line manage to explain?” Higher is better.

17 The whole story on one page

★ Key Takeaways

- We predict a number by **drawing a line** through past data: $\text{guess} = \text{start} + \text{rate} \times \text{fact}$ (the symbols are $\hat{y} = \theta_0 + \theta_1 x$).
- “Linear” just means **multiply each fact by its knob and add up**. More facts = more knobs.
- For any line we measure the **misses** (real – guess), **square** them so they don’t cancel, and **average** them. That score is the **MSE**.
- **Learning** = **finding the knobs that make the MSE smallest** — the bottom of a bowl. For linear regression the bowl has a single bottom, so we can’t get stuck.
- Reach the bottom two ways: a **one-shot formula** (great for few features) or a **down-hill walk** (gradient descent, great for big data; pick a step size, scale your features).
- Feed in **squared facts** to bend the line into curves — same method, richer ingredients.
- Avoid **too simple** (misses the pattern) and **too wiggly** (chases noise); land in the middle with more data, fewer ingredients, early stopping, or **regularisation** (Ridge keeps knobs small, Lasso switches useless ones off).
- Grade the result on *unseen* data with R^2 (1 is perfect, 0 is no better than the average).

Where this comes from: Bishop, *Pattern Recognition & Machine Learning*, Ch. 3; Andrew Ng’s lectures; Stanford CS229. Course slides M3 (Dr. Neha Vinayak, BITS Pilani WILP).